

INFO506 Td n°2

September 29, 2026

1 Gestion des erreurs en C Système

Comme nous l'avons vu en CM les appels système peuvent échouer mais ne provoque pas l'arrêt de votre processus **il est donc très important de gérer ces cas pour ne pas provoquer de réelles erreurs dans la suite vos programmes.**

1.1 Les codes d'erreur

Les erreurs sont traitées à travers des macros définies historiquement dans l'en-tête **errno.h** dans les sources modernes du noyau il est plus difficile de les localiser. On utilise donc plus simplement la commande système **errno**

```
errno -1 | less
```

Vous allez recréer cette commande en utilisant la famille de fonction **strerror**

```
char *strerror(int errnum);  
    const char *strerrorname_np(int errnum);  
    const char *strerrordesc_np(int errnum);
```

Les erreurs sont codées sur un octet vous pouvez donc afficher les erreurs grâce à une boucle de 0 à de 256. Selon le système le nombre de macro est variable, vous devez donc traiter les cas où le retour est **NULL**.

1.2 Mécanisme de propagation des erreurs

L'appel système lui même (code noyau) en cas de problème retourne la valeur négative du code par exemple **-EPERM**. Le wrapper de la LibC place (-) cette valeur **dans la variable globale errno**.

Par exemple pour traiter les erreurs produites par l'appel système open:

```
int open(const char *filename, int flags, int mode)
{
    int ret;
    asm volatile (
        "int $0x80"
        : "=a" (ret)
        : "a" (__NR_open),
          "b" (filename), "c" (flags), "d" (mode)
    );

    if (ret < 0) {
        errno = -ret;
        return -1;
    }
    return ret;
}
```

La variable `errno` est définie dans le fichier **myerrno.h** ainsi que les deux fonctions **strerror-name**, **strerrordesc**. Testez dans main les erreurs produites par le noyau.

2 Cycle de vie des processus sous Linux

Question 1 Réalisez un schéma (États/Transition) qui décrit les cycle de vie des processus. Donnez les nom des états, les éléments qui expliquent les transitions, les appels systèmes mis en jeu.

3 L'appel système fork

3.1 Résultat de l'appel

Question 1 Que va produire le code suivant:

```
#include<unistd.h>
#include<stdio.h>
void main(void){
    int pid=fork();
    if(pid){
        printf("\n\e[1;42m Pour le pere: pid = %d \e[0m\n", pid
    ) ;
    } else {
        sleep(10) ;
        printf("\n\e[1;41m Pour le fils: pid = %d \e[0m\n",
            pid) ;
    }

    while(1) ;
}
```

NB:

Ce code se trouve dans le fichier **E3_Q1_fork.c**

e[1;42m est un code pour modifier la couleur du terminal

Compilez le fichier

```
gcc -o E3_Q1_fork E3_Q1_fork.c
```

Lancer le en tâche de fond

```
$/E3_Q1_fork.c &
```

Question 2 Visualiser l'état des processus avec la commande ps.

```
ps -p <pid fils> -ocmd,stat,ppid
```

Question 3 Exécutez la commande **kill** pour arrêter le processus fils.

```
kill -9 <pid fils>
```

Relancez la commande ps et expliquer le changement d'état du processus fils.

Question 4 Compilez le fichier avec les options de débogage ci-dessous:

```
gcc -g -O0 -o E3_Q1_fork E3_Q1_fork.c
```

Nous allons utiliser gdb pour suivre successivement l'exécution du processus père puis du fils.

Pour visualiser l'exécution du processus père :

```
gdb E3_Q1_fork
(gdb) set detach-on-fork off
(gdb) b 5
(gdb) r
(gdb) p pid
(gdb) c
(gdb) exit
```

Pour visualiser l'exécution du processus fils :

```
gdb E3_Q1_fork
(gdb) set detach-on-fork off
(gdb) set follow-fork_mode child
(gdb) b 5
(gdb) r
(gdb) p pid
(gdb) c
(gdb) exit
```

3.2 Arborescence des processus

Question 5 Écrivez deux programmes qui utilisent une variable n pour générer les arborescences suivantes:

<pre>processus - fils-1 - fils-2 - fils-3 ... - fils-n</pre>	<pre>processus - fils-1 - fils-2 - fils-3 ... - fils-n</pre>
--	--

Question 6 Quelle sera l'arborescence générées par les programmes suivants

```
for(int i=0; i < 4; i++){  
    fork() ;  
}
```

4 L'appel système exec

L'appel système **execv** permet de remplacer le code d'un processus par celui d'un autre programme.

Son prototype est :

```
int execv(const char *path, char *const argv[]);
```

Question 1 Que se passe t'il en cas de succès de cet appel système ?

Question 2 Expliquez les modifications qui sont réalisées au niveau de la mémoire du processus qui réalise cet appel système.

Question 3 Codez un processus qui utilisera les appels système fork et exec pour exécuter la commande pstree -p PPID. PPID est le PID du père du processus initial que l'on obtient avec la fonction **getppid()** sous forme d'un entier que l'on peut transformer en chaîne de caractère avec la fonction **sprintf**.